# 밥상 차리기 — 재료 원가 vs 사먹을 때

기능 명세서

기후 식탁 · 2026-06-26

# 기능 명세서 — 밥상 "재료 원가 vs 사먹을 때" + 소비자 구매단위

작성일: 2026-06-26
대상: 기후 식탁 / "밥상 차리기" 탭 (기존 기능 확장)

## 1. 목적

현재 밥상 탭은 **재료 원가**(집에서 사다 해먹는 비용)만 보여준다. 사용자가 체감하는 또 다른 현실인 **"밖에서 사먹을 때(외식) 금액"**을 함께 보여주고, 재료 가격을 **소비자가 실제 사는 단위(포기·개·망·단·포대)**로 표시해 "내 장바구니/외식비가 이 날씨에 얼마가 되는가"를 더 와닿게 한다.

## 2. 현재 동작 → 변경점 요약

| 항목 | 현재 | 변경 후 |
|---|---|---|
| 총액 표시 | 재료 원가 1개 | **재료 원가** + **사먹을 때(외식)** 2개 |
| 원자재 표기 | 작물 합산 가격(원/kg 환산) | 작물별 **구매단위 가격**(예: 배추 1포기 5,400원) + 평년比 % |
| 음식별 | 가격 1개 | 재료 원가 + 외식가 |
| 데이터 | dishes.py 레시피 | + 작물 구매단위(UNITS), + 음식별 외식 단가 |

## 3. 가격 모델

### 3-1. 재료 원가 (집에서 해먹을 때) — 기존 유지

- 작물 재료비 = 모델 예측 **도매가(원/kg) × 분량(kg)**.
- 고정 재료(고기·두부·양념)는 정가 합산.
- 평년 기준은 BASELINE 기상으로 같은 방식 계산.
- (기존 `compute_meal`의 total/dish 값과 동일. 명칭만 `cost`로 정리.)

### 3-2. 소비자 구매단위 (포기·개·망·단·포대)

작물별 "사람들이 실제 사는 단위"와 환산 중량을 정의한다. 단위 가격은 **도매가(원/kg) × 단위중량**으로 계산(기존 앱 packPrice 관례와 동일, 별도 소매 마진 없음 → 과대표기 방지).

| 작물 | 단위 | 단위중량 | 평년 단위가(예시) |
|---|---|---|---|
| 배추 | 포기 | 2.5kg | ~5,400원 |
| 무 | 개 | 1.0kg | ~1,220원 |
| 사과 | 개 | 0.3kg | ~1,000원 |
| 쌀 | 20kg 포대 | 20kg | ~54,000원 |
| 시금치 | 단 | 0.4kg | ~1,320원 |
| 양파 | 망 | 1.5kg | ~1,670원 |

- 평년 단위가는 모델 평년 도매가로 런타임 계산(위는 8월 기준 예시).
- 원자재 분해에서 "배추 1포기 5,400원 · 평년比 +78%"처럼 **단위가 + 상승률**을

보여준다. (밥상 총액 원가는 분량 기준 그대로, 단위가는 체감용 표기)

### 3-3. 사먹을 때 (외식) — 신규

- 음식별 **외식 기준 메뉴가(1인분)**를 정의한다(평년 기준).

| 음식 | 외식 단가(평년) |
|---|---|
| 공기밥 | 1,000 |
| 김치찌개 | 8,000 |
| 된장찌개 | 8,000 |
| 제육볶음 | 9,000 |
| 배추김치(반찬 단품) | 3,000 |
| 무생채(반찬 단품) | 3,000 |
| 시금치나물(반찬 단품) | 3,000 |
| 양파장아찌(반찬 단품) | 2,000 |

- **기후 반영(전가)**: 외식가 = 외식 단가 + (해당 음식 재료 원가의 평년 대비

상승분). 식당도 재료비가 오른 만큼 천천히 메뉴가를 올린다는 현실을 반영.

- `eatout_predicted = menu_base + (cost_predicted - cost_baseline)`
- `eatout_baseline = menu_base`
- 평년에는 외식가 = 메뉴 기준가, 폭염 등에는 재료 상승분만큼 가산.
- 반찬은 메뉴가가 작아 외식가 상승 폭도 작다(현실적).

> 외식 단가·구매단위·환산중량은 모두 교육용 근사값이며 `backend/dishes.py` 한 곳에서 조정한다.

## 4. API 변경

**`GET /api/dishes`**

각 음식에 `eatout` (외식 기준 단가) 추가. (재료 구조는 기존과 동일)

**`GET /api/units`** **(신규, 또는 /api/dishes 응답에 포함)**

작물별 구매단위: `{ "배추": {"unit":"포기","kg":2.5}, ... }`

- 구현 단순화를 위해 `/api/dishes` 응답에 `units` 키로 함께 반환한다.

**`POST /api/meal`**

응답 구조 확장:

```
{
  "total": {
    "cost":   {"predicted": 13573, "baseline": 12680, "pct": 7},
    "eatout": {"predicted": 41253, "baseline": 40000, "pct": 3}
  },
  "dishes": [
    {"id":"kimchi_jjigae","name":"김치찌개",
     "cost":{"predicted":4400,"baseline":3700,"pct":19},
     "eatout":{"predicted":8700,"baseline":8000,"pct":9},
     "ingredients":[ ... 기존과 동일 ... ]}
  ],
  "ingredientTotals": [
    {"crop":"배추","unit":"포기","unitKg":2.5,
     "unitPrice":{"predicted":9612,"baseline":5400,"pct":78},
     "predicted":1155,"baseline":648,"pct":78,
     "usedIn":["김치찌개","배추김치"]}
  ]
}
```

- `total.cost` / `total.eatout` 로 총액 2종.
- 각 음식에 `cost` / `eatout` .
- `ingredientTotals[].unit/unitKg/unitPrice` 로 구매단위 가격(평년·예측·%).
- 기존 `total.predicted` 같은 평면 키는 `total.cost.predicted` 로 이동(프론트 함께 수정).

## 5. UI 변경 (frontend)

### 결과 패널 상단 — 총액 2개

```
재료 원가(집밥)      13,573원   +7%
사먹을 때(외식)      41,253원   +3%
```

- 두 줄/두 칼럼로 나란히. 원가는 기존 강조 스타일, 외식가는 보조 스타일.
- 인사이트 문구는 기존 유지(원자재 상승 주도 작물).

### 음식별

- 각 음식 행에 원가 / 외식가 함께 표기(예: `김치찌개 원가 4,400 · 외식 8,700 +19%`).

### 원자재(기후 작물) 분해

- 기존 막대에 **구매단위 가격**을 함께: `배추 1포기 9,612원 (평년 5,400) · +78%`.
- 막대 길이는 기존처럼 상승분 비례.

## 6. 엣지 케이스 / 검증

- 빈 밥상: cost/eatout 모두 0, pct 0.
- 평년 시나리오: cost.pct ≈ 0, eatout.pct ≈ 0(전가분 0).
- 알 수 없는 음식 id: 스킵(기존과 동일).
- 외식 전가분 음수(풍작): 외식가가 메뉴 기준가보다 소폭 하락 허용.
- test_api.py 확장:
- `/api/dishes`에 `eatout` 및 `units` 존재.
- `/api/meal` 응답에 `total.cost` / `total.eatout`, `ingredientTotals[].unit/unitPrice` 존재.
- 폭염 시 cost.pct>0 이고 eatout.predicted ≥ eatout.baseline.
- 평년 시 cost.pct≈0 및 eatout.pct≈0.
- 기존 29개 검증 회귀 없음(평면 키 → 중첩 키 변경에 따른 테스트 수정 포함).

## 7. 범위 밖 (YAGNI)

- 지역별 외식가 차이, 인건비/임대료 별도 모델, 수량 스텝퍼 UI, 영수증 출력.
- 소매 유통마진 별도 계수(도매×단위중량 관례 유지).

## 8. 튜닝 포인트(검토 시 확인)

1. 외식 단가 값(4-3 표) — 가게 기준 조정 가능.
2. 구매단위·환산중량(3-2 표) — 기존 UNIT_PACK과 동일하게 둠.

3. 외식가 기후 전가 방식 — "재료 상승분 전액 가산" 채택(대안: 배수·부분전가).